

Your FYP Title Here

Group Member 1 (Roll Number)

Group Member 2 (Roll Number)

Group Member 3 (Roll Number)

Supervised by: **Supervisor's Name**

Submitted in partial fulfillment of the requirements for the degree of
BS Computer Science

Department of Computer Science
Government College University Lahore

Abstract

Write your 150-250 word project summary here. State the problem, your proposed software solution, the technologies used, and the final outcome of the project.

Contents

Abstract	iii
1 Introduction	1
1.1 Project Overview	1
1.2 Problem Statement	1
1.3 Project Objectives	1
2 Requirement Specification	3
2.1 Functional Requirements	3
2.1.1 User Management & Roles	3
2.1.2 Catalog & Pricing	4
2.1.3 Matchmaking & Discovery	4
2.1.4 Scheduling & Calendar	4
2.1.5 Job Execution & Rescheduling	5
2.1.6 Financial Ledger & Wallet	5
2.1.7 Rating & Reviews	6
2.1.8 Payment Gateway Integration	6
2.1.9 Support & Dispute System	6
2.1.10 Dispute Resolution & Refunds	6
2.1.11 Notification Engine	7
2.1.12 Admin Dashboard	7
2.1.13 Address and Mapping Services	7
2.1.14 Real-Time Technician Tracking	8
2.2 Non-Functional Requirements	8
2.2.1 Security & Privacy	8
2.2.2 Performance & Responsiveness	8
2.2.3 Reliability & Availability	9

2.2.4	Scalability	9
2.2.5	Maintainability & Extensibility	9
2.2.6	Usability & Accessibility	9
2.3	Use Case Models	10
2.3.1	User Onboarding & Account Management	10
2.3.2	Service Discovery & Booking (Matchmaking)	12
2.3.3	Job Execution & Rescheduling	13
2.3.4	Financial Ledger & Wallet Management	15
2.3.5	AI Support, Disputes & Admin Operations	16
2.4	System Constraints	18
2.4.1	Payment Gateway & Sandbox	18
2.4.2	Network & Connectivity	18
2.4.3	Hardware & Permissions	18
2.4.4	Platform Support	18
2.4.5	API Limits & Dependencies	19
3	Project Design	21
3.1	System Architecture	21
3.1.1	Architectural Philosophy: The “Clean” Approach	21
3.1.2	Detailed Flutter Client Architecture	23
3.1.2.1	Presentation Layer	23
3.1.2.2	Domain Layer: The Business Logic “Brain”	23
3.1.2.3	Data Layer: The Implementation Tier	24
3.1.3	Detailed Django Backend Architecture	25
3.1.4	Data Propagation and Request Lifecycle	27
3.2	Behavioral Modeling	28
3.2.1	Data Flow Analysis	28
3.2.2	Sequence Diagrams	30
3.2.2.1	The Multi-Phase Job Lifecycle	30
3.2.2.2	Real-Time Tracking	30

3.2.2.3	Technician Top-Up	31
3.2.2.4	Dispute Resolution	32
3.3	Database Design	32
3.3.1	Master Entity Relationship Diagram (ERD)	32
3.3.2	Modular Data Schemas	33
3.3.2.1	Identity and Role Management	33
3.3.2.2	Technician Profiles and Scheduling	34
3.3.2.3	Service Catalog and Marketing	34
3.3.2.4	Bookings and Field Operations	35
3.3.2.5	Financial Ledger and Support	36
3.4	User Interface Design	37
3.4.1	Design Principles	37
3.4.2	User Experience (UX) Strategy	38
4	Project Design	39
4.1	System Architecture	39
4.2	Database Design	39
4.3	User Interface Design	39
5	Conclusion and Future Work	41
5.1	Conclusion	41
5.2	Limitations	41
5.3	Future Work	41
	Bibliography	43

List of Figures

2.1	Use Case Diagram: User Onboarding & Account Management	10
2.2	Use Case Diagram: Service Discovery & Booking	12
2.3	Use Case Diagram: Job Execution & Rescheduling	13
2.4	Use Case Diagram: Financial Ledger & Wallet Management	15
2.5	Use Case Diagram: AI Support, Disputes & Admin Operations	16
3.1	Unified System Architecture	22
3.2	The 4-Step Error Pipeline	25
3.3	Data Propagation Flow from UI to Database	27
3.4	Context Diagram (Level 0 DFD)	29
3.5	Level 1 DFD: Internal Process Decomposition	29
3.6	Multi-Phase Job Execution Sequence	30
3.7	Real-Time GPS Broadcasting Sequence	31
3.8	Secure Wallet Top-Up Sequence	31
3.9	AI-Integrated Dispute and Refund Pipeline	32
3.10	Master Entity Relationship Diagram	33
3.11	Identity and Access Schema	34
3.12	Technician Domain Schema	34
3.13	Catalog and Promotion Schema	35
3.14	Operational Booking Schema	36
3.15	Financial and Support Schema	37

List of Tables

Chapter 1

Introduction

1.1 Project Overview

Provide a high-level overview of the software or system you are building.

1.2 Problem Statement

What specific problem does this project solve?

1.3 Project Objectives

List the main goals of your project.

Chapter 2

Requirement Specification

2.1 Functional Requirements

2.1.1 User Management & Roles

- The system will use a phone-number based login for all users..
- When a user enters their number, the system will send a One-Time Password (OTP) via SMS.
- The OTP will expire after 30 seconds. Users can request a new code if the timer runs out.
- If the user already has an account, they will be logged in and sent to the Home Screen after the OTP is verified.
- If it's a new user, the system will ask for their "Full Name" to create an account before moving to the Home Screen.
- Customers can apply to become technicians by submitting their CNIC number, profile photo, CNIC photos, specific skills, and their labor rate.
- All technician applications must be manually approved by an admin.
- Once approved, the technician must set their working hours, base location, and how far they want to travel (travel radius).
- Technicians who have not been approved will not show up in search results.
- The app will allow users to switch between "Customer" and "Technician" modes within the same account.

2.1.2 Catalog & Pricing

- The service catalog will be organized into main categories (e.g., Plumbing) and specific tasks (e.g., Tap Repair).
- Main services will include a standard inspection fee and an estimated time for the first visit.
- The system will support fixed prices for standard tasks, skipping the quote process for those specific jobs.
- For jobs that aren't fixed-price, technicians can set their own labor rates within a range set by the admin.
- The catalog will include a "Temporary Patch" option with a price cap, or technicians can add it as a custom line item.

2.1.3 Matchmaking & Discovery

- The app will have a search bar that uses keyword similarity to help user finds services and sub services based on their query.
- The home screen will show active promotions, service categories, and popular fixed-price jobs in a single feed.
- The system will only show technicians who are within range of the customer's location based on the technician's set travel radius.
- Technicians will be ranked using a rating formula to balance their star ratings with the number of jobs they have completed.

2.1.4 Scheduling & Calendar

- The system will calculate technician time slots by checking the technician's existing bookings and personal time off against their weekly schedule.
- Booking time block will be sized automatically based on the duration of selected service or sub service.

- The system will add extra time between consecutive booking for the technician to travel between locations.

2.1.5 Job Execution & Rescheduling

- The system uses a two-phase booking process: first for the initial inspection and then for the actual repair.
- After the inspection, the technician will use an in-app tool to list the required sub-services, which the customer must approve before work starts.
- Technicians can change a fixed-price job back to a diagnostic status if the job turns out to be more complex than expected.
- If a job takes too long, technicians can offer a “temporary patch” for a small fee and then propose a reschedule follow up visit for the full job.
- When a follow-up is proposed, the time slot will be marked as “Pending” on the technician’s calendar until the customer accepts it.
- Follow-up bookings will store the original booking ID as a “Parent ID” so the job history remains linked.

2.1.6 Financial Ledger & Wallet

- The system will keep a record of all transactions (deposits, fees, and promotions) in a wallet ledger that prevents duplicate entries.
- The platform commission will be deducted from the technician’s virtual wallet after a job is finished, while the customer pays the technician in cash.
- Technician must maintain a minimum wallet balance to prevent his account from locking and continue receiving new job requests.
- If a customer uses a platform discount, the system will add the discounted fee to the technician’s wallet to cover the difference.
- Technicians can request to withdraw their wallet balance, which will be held pending until an admin manually send the balance and approve it.

2.1.7 Rating & Reviews

- Customers can only leave a review if the booking was actually completed (not cancelled).
- New ratings will automatically update the technician's overall average in the database.
- If a job is closed as a "Temporary Patch," the rating prompt will be skipped until the final repair is finished.

2.1.8 Payment Gateway Integration

- The system will use webhook to verify payment status, to ensure that the transactions are recorded even if the app is closed or the user lose connection.
- The system will use a unique transaction id as an idempotent key for all incoming payment notifications to prevent duplicate wallet updates

2.1.9 Support & Dispute System

- The system will use a chat bot for common user queries and initial issue reporting.
- The chatbot will use NLP to provide information on platform fees, booking rules, and the two-phase payment structure
- The assistant will have access to the user booking history and the phase 2 quote to provide relevant support.
- Users can submit photos and text descriptions through the chat interface to initiate a dispute.
- The system will automatically generate a dispute ticket on the information processed by the chat bot.

2.1.10 Dispute Resolution & Refunds

- The system will track the refund requests.

- Admin can update the refund status from pending to transferred (once he sends the external wallet payments for the disputed amount) .
- In the case of refund, the system will deduct the disputed amount from the technician 's wallet.
- The system will restrict a technician from accepting new jobs if his wallet balance falls below the minimum threshold.

2.1.11 Notification Engine

- The system will support deep-linking in push notifications to route users to specific screens, such as Quote Approval or Dispute Status.
- Real-time alerts will be sent for:
 - Low wallet balance (technicians).
 - Changes in dispute ticket status.
 - New reschedule requests and their outcomes.

2.1.12 Admin Dashboard

- The portal will allow administrators to review, approve or reject technician registration documents.
- The dashboard will include tools to manage promotions, including usage limit and expiration date.
- The dashboard will let administrators handle all active disputes and technician withdrawal requests in one view.

2.1.13 Address and Mapping Services

- The system will integrate a Geocoding service to convert text addresses to latitude and longitude.
- The booking checkout process will feature a map where users can manually select their location for service delivery.

2.1.14 Real-Time Technician Tracking

- The technician flutter app will send live coordinates to our backend during the transit phase.
- The system will use cache to broadcast the coordinates to the customer app.
- The customer app will show the technician's movements on an interactive map using the Google Map SDK

2.2 Non-Functional Requirements

2.2.1 Security & Privacy

- The system will use Token-based authentication to manage user sessions, ensuring that sessions expire after a period of inactivity.
- We will use role based access on all api endpoints so that users can only access data relevant to their current role.
- All incoming webhooks must be verified before updating the user's wallet.

2.2.2 Performance & Responsiveness

- The search and matchmaking results should be returned in less than 800ms under normal load.
- The WebSocket server should broadcast GPS updates every 3 seconds with a maximum latency of 500ms.
- The AI chatbot should find and display the user's history and booking context in less than 1.5 seconds.
- The map interface should maintain a smooth frame rate (60 FPS) while animating the technician's location.

2.2.3 Reliability & Availability

- The core platform services (matching, scheduling, and catalog) will aim for 99.9% availability.
- All payment and wallet operations must be idempotent to prevent duplicate charges or deposits during network retries.
- The wallet system will use locking in the database to ensure that balance updates happen sequentially and accurately.

2.2.4 Scalability

- The backend will be stateless to support horizontal scaling as the number of users grows.
- The WebSocket/telemetry servers should be able to handle at least 1000 simultaneous connections without a performance drop.

2.2.5 Maintainability & Extensibility

- The service catalog will be managed via the admin dashboard, allowing admins to add or edit categories and promotions without requiring a new app store release.
- The system will maintain an audit log of all admin actions (approvals, dispute overrides, and withdrawals).
- The GPS tracking service will be decoupled from the main database to ensure high-frequency updates don't slow down the booking system.

2.2.6 Usability & Accessibility

- Switching between the Customer and Technician interface should take less than 1 second and should not require a re-login.

2.3 Use Case Models

2.3.1 User Onboarding & Account Management

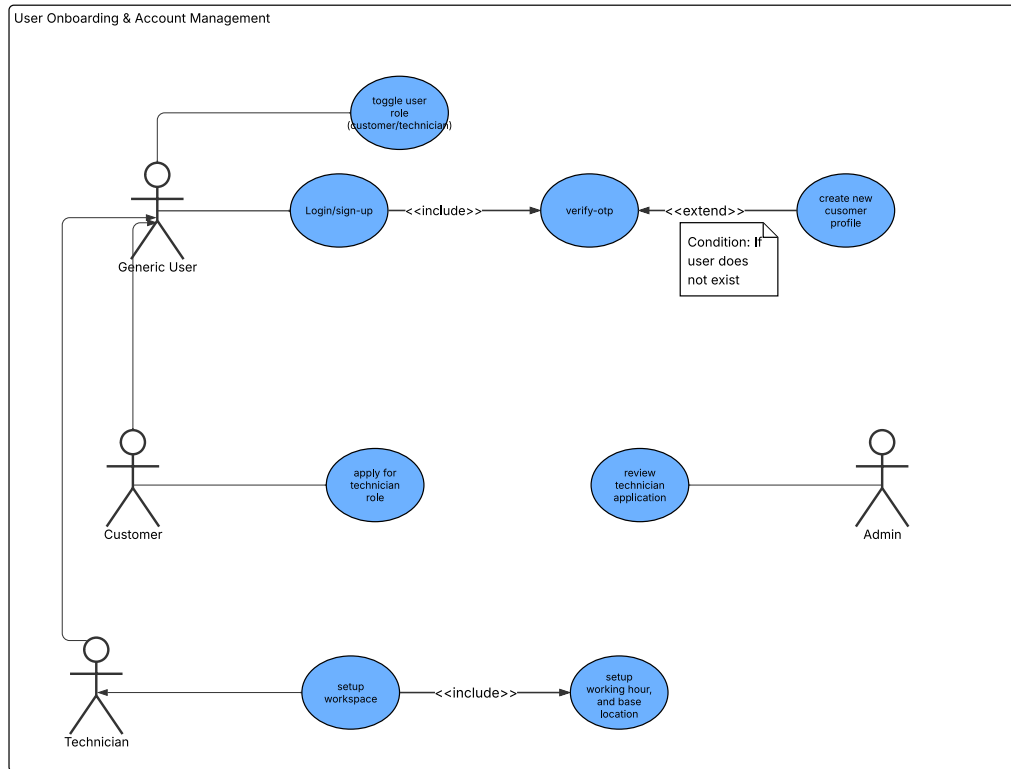


Figure 2.1: Use Case Diagram: User Onboarding & Account Management

Use Case UC01: Authentication & Role Toggling

- **Actor:** User
- **Description:** The process for a user to login into the customer view, system verification, create a base profile, switch to technician view.
- **Preconditions:**
 - The application is installed on the user's device.
- **Post Conditions:**
 - The user is authenticated and viewing the correct dashboard based on their toggle.
- **Main Flow:**
 1. The User enters their mobile number for Login/Sign-up.
 2. The System sends a One-Time Password (OTP) with an expiry of 30 seconds.
 3. The User inputs the OTP to verify their identity.

4. Extension: If the User does not exist in the database, the System asks for their full name and creates a new user in the database.
5. Once logged in, the User can switch between Customer and Technician view using a toggle.

Use Case UC02: Technician Onboarding & Workspace Setup

- **Actor:** Customer, Admin, Technician
- **Description:** The process of Customer applying to create a Technician profile, getting approved by Admin, and start configuring their service workspace.
- **Preconditions:**
 - The user is logged in and has a default Customer profile.
- **Post Conditions:**
 - The Technician is fully onboarded, and their profile is visible in customer search results during their active hours.
- **Main Flow:**
 1. The user applies for the Technician role by submitting required details.
 2. The Admin reviews the technician application from their dashboard and approves it.
 3. The user's account is updated to allow toggling to the Technician view from their default Customer view.
 4. The Technician navigates to set up their workspace.
 5. The Technician set their working hours and base location.

2.3.2 Service Discovery & Booking (Matchmaking)

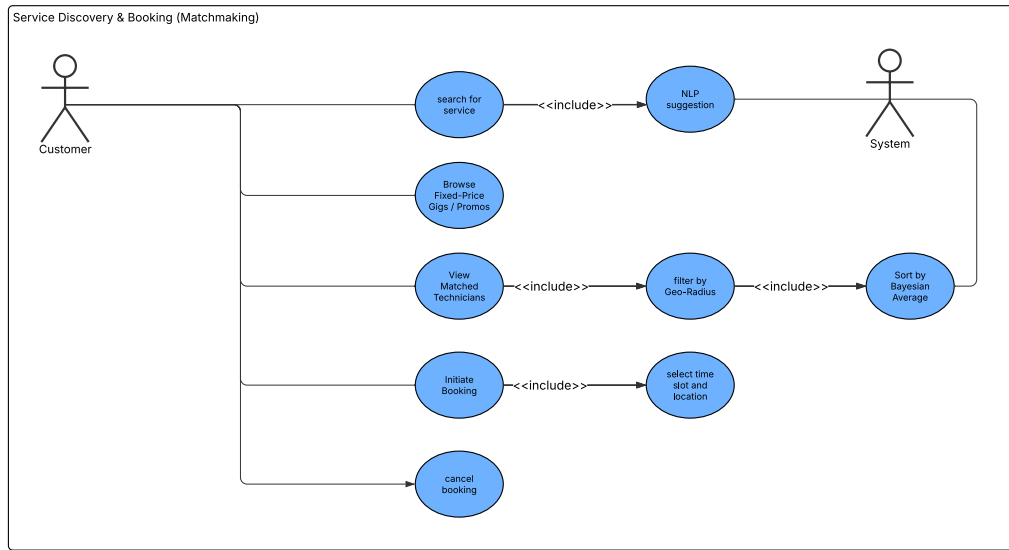


Figure 2.2: Use Case Diagram: Service Discovery & Booking

Use Case UC03: Search and Match Technicians

- **Actor:** Customer, System
- **Description:** The customer searches for a specific service and system filter and ranks top Technicians near him.
- **Preconditions:**
 - Customer is logged in with location services enabled.
- **Post Conditions:**
 - The customer is presented with an optimized, localized list of available technicians.
- **Main Flow:**
 1. The Customer inputs a query to search for a service.
 2. The search bar processes user query and matches it with our predefined services and sub services using NLP-based keyword matching.
 3. The Customer click on a service
 4. The Customer views a list of Matched Technicians.
 5. The System filters these technicians by a specific Geo-Radius relative to the customer's location.
 6. The System sorts the filtered list using a rating formula (Bayesian average), to ensure the technician with many good reviews appears above those with only one or two five star ratings.
 7. Alternative Flow: The Customer bypasses the search and tries to view technicians by clicking the aggregated services, fixed price gigs or promotional offer on dashboard.

Use Case UC04: Booking Management

- **Actor:** Customer
- **Description:** The process of finalizing a service appointment or canceling it.
- **Preconditions:**
 - The customer has selected a specific technician profile.
- **Post Conditions:**
 - A booking is successfully scheduled in the system.
- **Main Flow:**
 1. The Customer initiates the booking process.
 2. The Customer selects a preferred time slot and confirms the service location.
 3. The System creates a pending booking and notifies the technician.
- **Alternate Flow:**
 - **AF1: Cancel Booking:** The Customer cancels a pending or upcoming booking before the technician arrives, and the System releases the reserved time slot.

2.3.3 Job Execution & Rescheduling

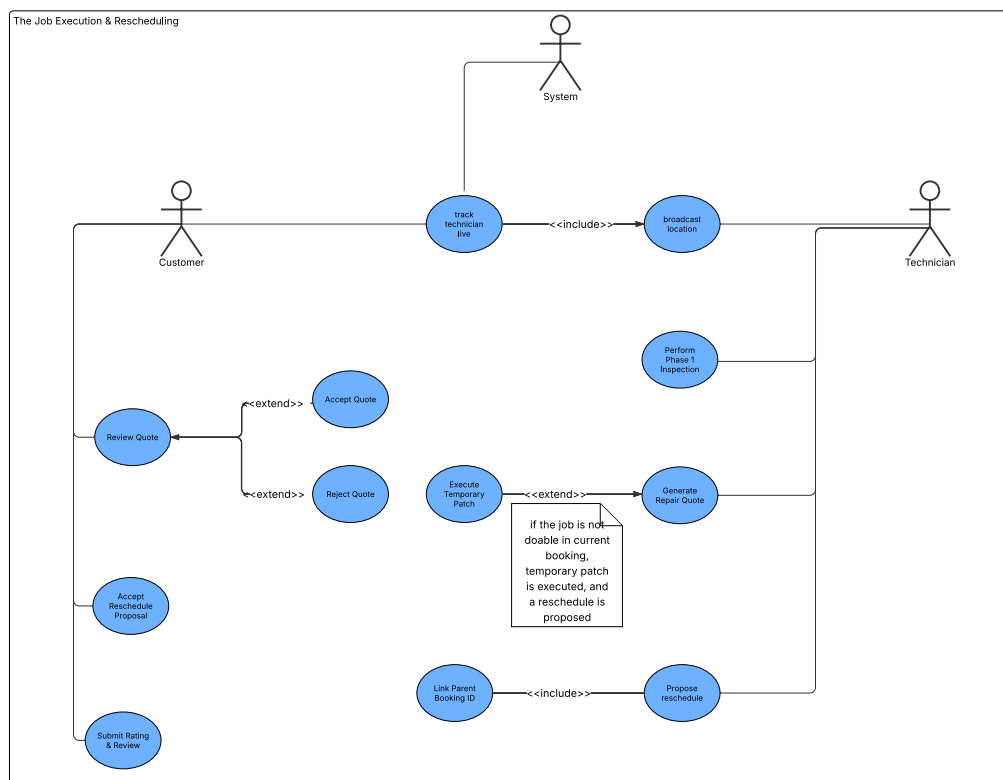


Figure 2.3: Use Case Diagram: Job Execution & Rescheduling

Use Case UC05: Live Tracking & Phase 1 Inspection

- **Actor:** Customer, Technician, System
- **Description:** The customer tracks the live technician arrival and the technician inspects the issue.
- **Preconditions:**
 - An active booking is scheduled for the current time slot.
- **Post Conditions:**
 - The technician is on-site and has diagnosed the core issue.
- **Main Flow:**
 1. The System initiates live tracking.
 2. The Technician's flutter app sends its coordinates to our backend every few seconds, so the Customer can see their progress on the map.
 3. The Customer tracks the technician live on their map interface.
 4. When the technician arrives, he performs the Phase 1 inspection of the issue for diagnosis.

Use Case UC06: Complex Job Rescheduling & Quoting

- **Actor:** Technician, Customer
- **Description:** The workflow quotation of jobs and also those jobs which can not be completed in time.
- **Preconditions:**
 - The Technician has done the phase 1 inspection.
- **Post Conditions:**
 - The job is either completed or rescheduled.
- **Main Flow:**
 1. The Technician generates a detailed Repair Quote which includes different sub services for the permanent fix.
 2. The Customer reviews the detailed quote.
 3. The Customer accepts the quote (or rejects it).
 4. Upon acceptance, the system updates the booking total; if rejected, only the base inspection fee is billed.
 5. Upon final completion of the job, the Customer submits a rating and review.
- **Alternate Flow:**
 - **AF1:** If the job is not possible, the Technician generates a quote with a temporary patch , and a reschedule proposal. The customer accepts it or rejects it. If accepted, the System creates a new follow-up booking that references the ID of the current booking for record.

2.3.4 Financial Ledger & Wallet Management

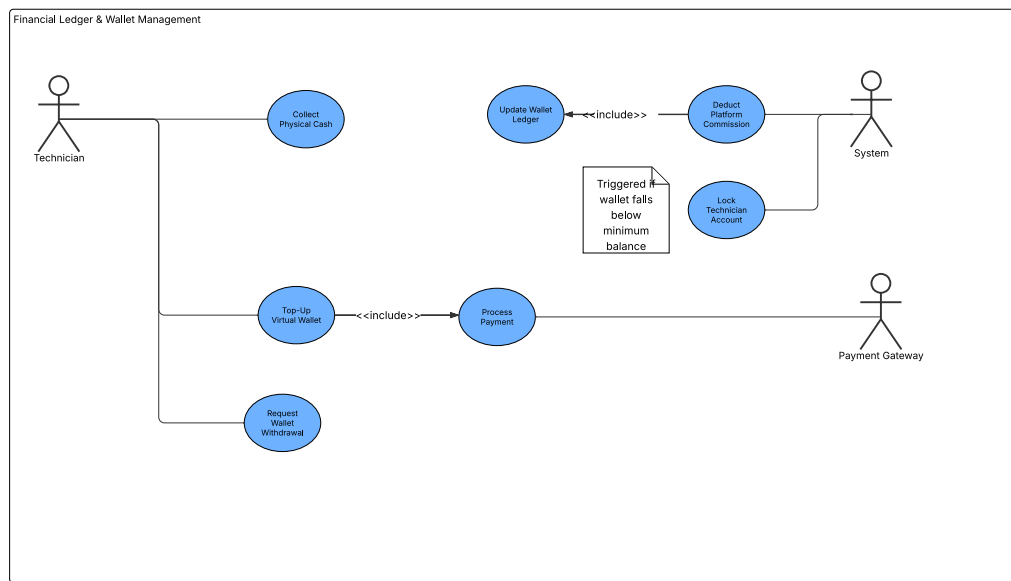


Figure 2.4: Use Case Diagram: Financial Ledger & Wallet Management

Use Case UC07: Cash Collection & Automated Ledger Maintenance

- **Actor:** Technician, System
- **Description:** The process of deducting the platform commission from technician wallet once the job is completed.
- **Preconditions:**
 - A job has been completed where the payment method was cash.
- **Post Conditions:**
 - The platform commission is settled, and the technician's account status is updated based on their remaining virtual balance.
- **Main Flow:**
 1. The Technician collects physical cash from the Customer.
 2. Our backend automatically calculates the commission and subtracts it from the Technician virtual wallet.
 3. Extension: If deducting the commission causes the technician wallet to fall below the minimum balance threshold, the system locks the technician from accepting new jobs.

Use Case UC08: Wallet Top-Up and Withdrawal

- **Actor:** Technician, Payment Gateway

- **Description:** The process for a technician to top up their virtual wallet via a payment gateway or request to withdraw their earned balance. Admin review and processing of withdrawal requests is covered separately under UC10 (Core Admin Operations).
- **Preconditions:**
 - Technician has an active account.
- **Post Conditions:**
 - The technician’s virtual wallet balance is successfully updated.
- **Main Flow:**
 1. The Technician initiates a Top-Up to their Virtual Wallet.
 2. The system routes the transaction to the Payment Gateway to process the payment.
 3. The Payment Gateway confirms success, and the system updates the wallet ledger.
 4. Alternative Flow: The Technician submits a Wallet Withdrawal Request by providing their bank or wallet details. The system creates a pending request for the Admin to review and manually process.

2.3.5 AI Support, Disputes & Admin Operations

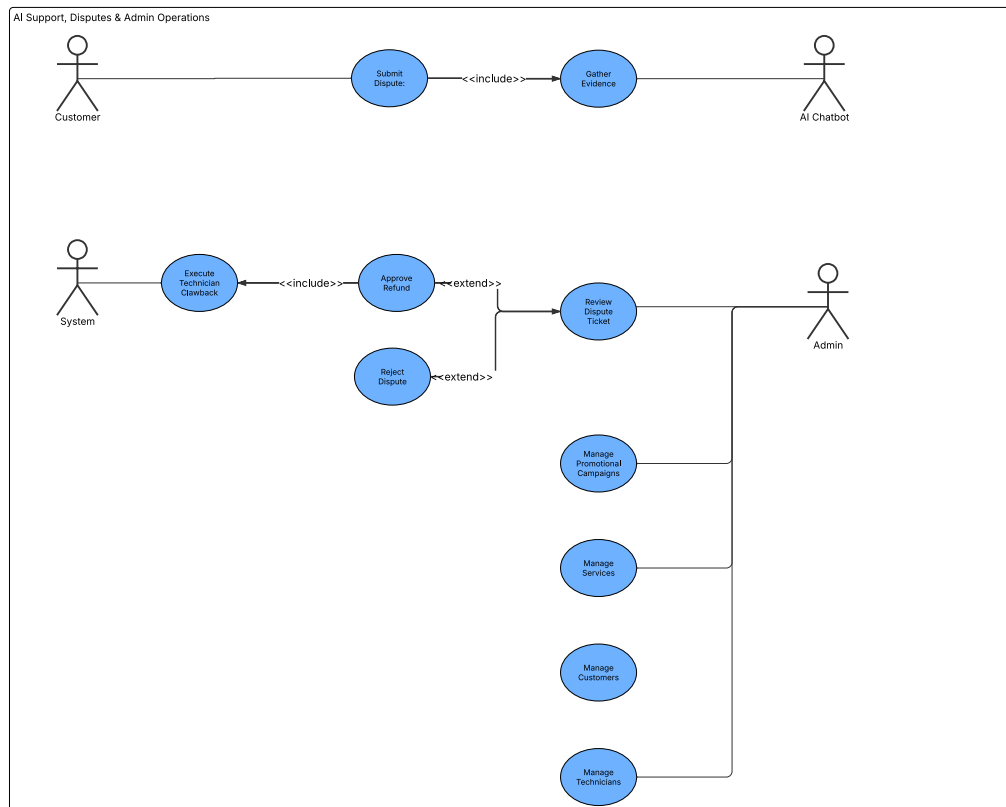


Figure 2.5: Use Case Diagram: AI Support, Disputes & Admin Operations

Use Case UC09: Dispute Resolution & Clawbacks

- **Actor:** Customer, AI Chatbot, Admin, System
- **Description:** The end-to-end process of a customer raising a complaint, AI gathering evidence and the admin reviewing the complaint.
- **Preconditions:**
 - A service has been completed and payment has been processed.
- **Post Conditions:**
 - The dispute is closed, the customer is notified, and ledger balances are adjusted if a clawback occurred.
- **Main Flow:**
 1. The Customer submits a Dispute regarding a completed service.
 2. The AI Chatbot starts a conversation with the Customer to ask for a photo of the work and description of the problem.
 3. The system compiles the data and creates a Dispute Ticket.
 4. The Admin reviews the Dispute Ticket.
 5. The Admin makes a decision to either Reject the Dispute or Approve a Refund.
 6. Extension (If Approved): The customer is notified to provide his bank details, and the admin manually sends the refunds to Customer, and the System reverses the commission and subtracts the disputed amount from the Technician wallet.

Use Case UC10: Core Admin Operations

- **Actor:** Admin
- **Description:** The daily operational tasks performed by platform administrators.
- **Preconditions:**
 - Admin is logged into the administrative backend.
- **Post Conditions:**
 - Platform data, services, and user statuses are updated in the central database.
- **Main Flow:**
 1. Admin navigates to the management dashboard.
 2. Admin can select “Manage Promotional Campaigns” to create or edit platform-wide discounts.
 3. Admin can select “Manage Services” to add, update, or remove service categories and base prices.
 4. Admin can select “Manage Customers” to review user accounts, ban users, or handle account queries.
 5. Admin can select “Manage Technicians” to review performance metrics, handle lockouts.

2.4 System Constraints

2.4.1 Payment Gateway & Sandbox

- **JazzCash Sandbox:** We are using the JazzCash Sandbox for testing because getting a live Merchant ID requires formal business registration and documents that are not available for this academic project.
- **Production setup:** The backend API and webhook logic are fully implemented. For deployment, we would only need to replace the sandbox credentials with a real Merchant ID and Secret Key in the settings.

2.4.2 Network & Connectivity

- **Internet Requirements:** The app needs a stable 4G or Wi-Fi connection. It will not work offline because the OTP login, GPS tracking, and our Django backend need to stay synced constantly.
- **Latency:** Real-time map updates depend on the network speed. In areas with weak signals, the technician's marker might jump on the customer's screen.

2.4.3 Hardware & Permissions

- **GPS Module:** The technician's phone must have a working GPS.
- **Permissions:** The app needs access to the camera and file storage. This is required for uploading CNIC photos during registration.

2.4.4 Platform Support

- **Android:** The Flutter app is optimized for Android 8.0 and above. It might run on older phones but Google Map SDK needs sufficient Processor and RAM to work smoothly.
- **Admin Portal:** The admin dashboard is designed for desktop browsers (Chrome,

Firefox, etc.). It is not mobile-friendly.

2.4.5 API Limits & Dependencies

- **Google Maps Quotas:** We are using the free tier of the Google Maps and Geocoding APIs. If the daily limits are hit during testing, the map and address search features will stop working until the quota resets.
- **Twilio SMS Gateway:** User authentication depends on Twilio. If Twilio goes down, OTPs will not be sent, and users won't be able to log in

Chapter 3

Project Design

3.1 System Architecture

The Karigar System is architected as a high-performance, decoupled, and scalable platform that uses modern three-tier architecture. This design prioritises the principle of Separation of Concerns. It ensures that the user interface, business logic and data layer can evolve and be maintained independently without having to look at every piece of the system.

3.1.1 Architectural Philosophy: The “Clean” Approach

The system is divided into two primary environments: the Flutter Client and the Django Server. They interact with one another using strictly defined JSON API Contract, which assure that the frontend and backend remain stateless and highly interoperable.

- **Decoupling:** By isolating business roles in a dedicated Domain layer, the system becomes independent of external entities and frameworks.
- **Testability:** The architecture allows for unit testing of business logic without invoking the UI or requiring a live database.
- **Scalability:** The stateless nature of the Django backend allows for horizontal scaling to support an increasing number of concurrent technicians and customers. Concurrently the clean architecture on the client side allows it to scale its feature set indefinitely without increasing architectural debt.

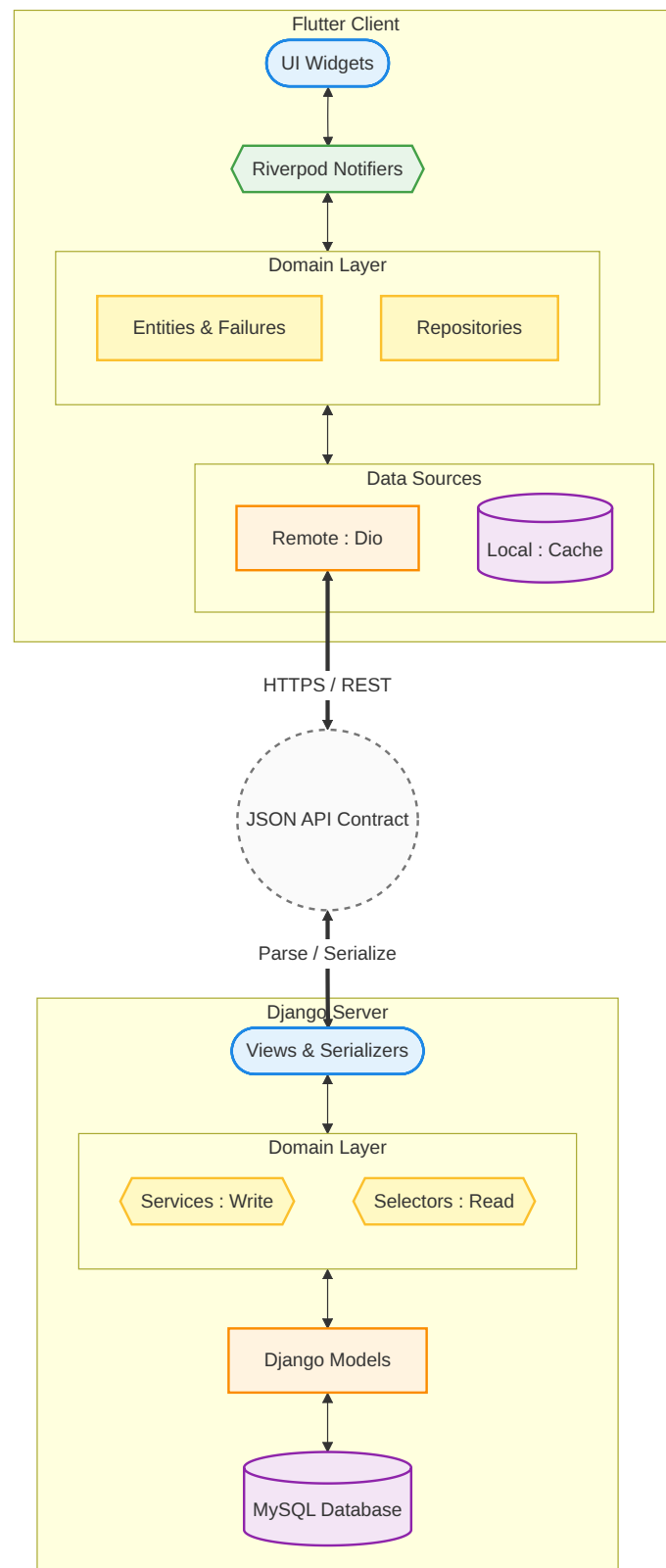


Figure 3.1: Unified System Architecture

3.1.2 Detailed Flutter Client Architecture

The frontend environment follows a layered approach where each feature is encapsulated within its own directory containing three specific layers: Presentation, Domain, and Data. This approach allows for a unidirectional flow of data which makes the system highly predictable and easier to debug.

3.1.2.1 Presentation Layer

This layer is responsible for rendering the interface. It manages the immediate visual state. It is designed to be reactive, meaning the UI automatically updates whenever the underlying data changes.

- **UI Widgets:** These are consumer stateful components that render data. They do not contain business logic. They watch for state changes and trigger action through a notifier.
- **State Management (Riverpod Notifiers):** The system uses modern Riverpod AsyncNotifiers to handle asynchronous state. This controller manages the lifecycle of the request transitioning the UI through Loading, Data, and Error state.

3.1.2.2 Domain Layer: The Business Logic “Brain”

The Domain Layer is the most critical part of the architecture. It is written in pure Dart code, meaning it has zero dependencies on Flutter or external API.

- **Entities:** These are the plain, immutable data objects that represent the core business concepts of the application.
- **Use Cases:** Every specific action a user can perform is encapsulated in a dedicated Use Case class. This “single-responsibility” approach makes the business logic highly testable and easy to maintain.
- **Repository Contracts:** This layer defines abstract interfaces that specify what data is needed. The Domain Layer does not care where the data comes from (a database or a server); it only defines the “contract” that other layers must follow.

- **Sealed Failures:** Instead of using generic errors, the system uses Sealed classes to represent specific typed failures. The UI layer just needs a switch condition on Sealed class to show the type of error encountered.

3.1.2.3 Data Layer: The Implementation Tier

The Data Layer is responsible for fulfilling the contract defined by the Domain layer; it handles the complexity of network communications and local storage.

- **Data Sources:** The system differentiates between Remote Sources, which communicate with the Django API, and Local Sources, which persist data on the device using SharedPreferences.
- **Data Models (DTOs) & Mapping:** To shield the application from backend changes, the system uses Data Transfer Objects (DTOs) for JSON parsing. A Mapping process then converts these into Domain entities before they are passed back.
- **The 4-Step Error Pipeline:** Every repository implementation follows a mandatory pipeline to catch low-level technical exceptions (like a `SocketException`) and transform them into the high-level Sealed Failures used in the Domain Layer.

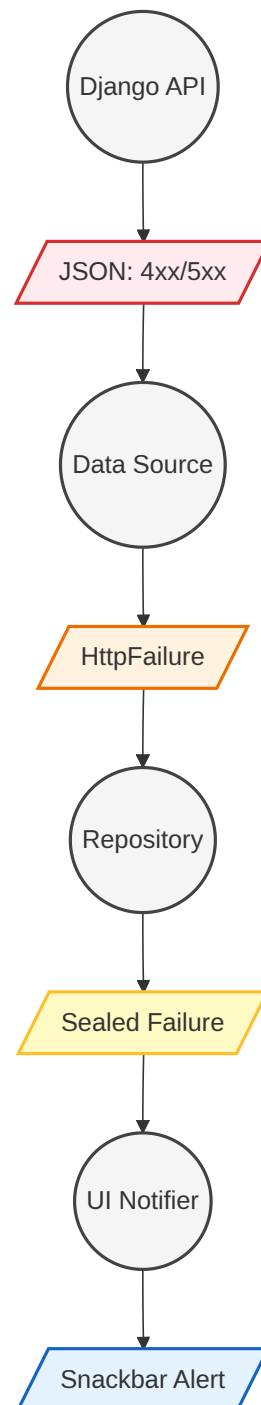


Figure 3.2: The 4-Step Error Pipeline

3.1.3 Detailed Django Backend Architecture

The server side infrastructure is built using the Django Rest Framework (DRF), which uses a domain-driven approach to ensure data integrity and high performance.

- **Views & Serializers:** This entry layer manages incoming HTTP requests, validating the API JSON contract and transforming the complex model into structured JSON for the client.
- **Domain Layer (Service-Selector Pattern):** The backend logic is divided into two categories for architectural clarity:
 - **Services (Write Operations):** These handle data mutations and complex business rules such as processing job bookings and updating wallets.
 - **Selectors (Read Operations):** These are optimized for high efficiency data read and complex queries such as searching for top technicians and ranking them.
- **Data Tier:** The Django Models (ORM) provide an abstraction over the MySQL Database. We just call Python functions that use ORM under the hood for complex SQL queries.

3.1.4 Data Propagation and Request Lifecycle

To justify the complexity of this architecture, it is essential to trace the lifecycle of a data request through the system.

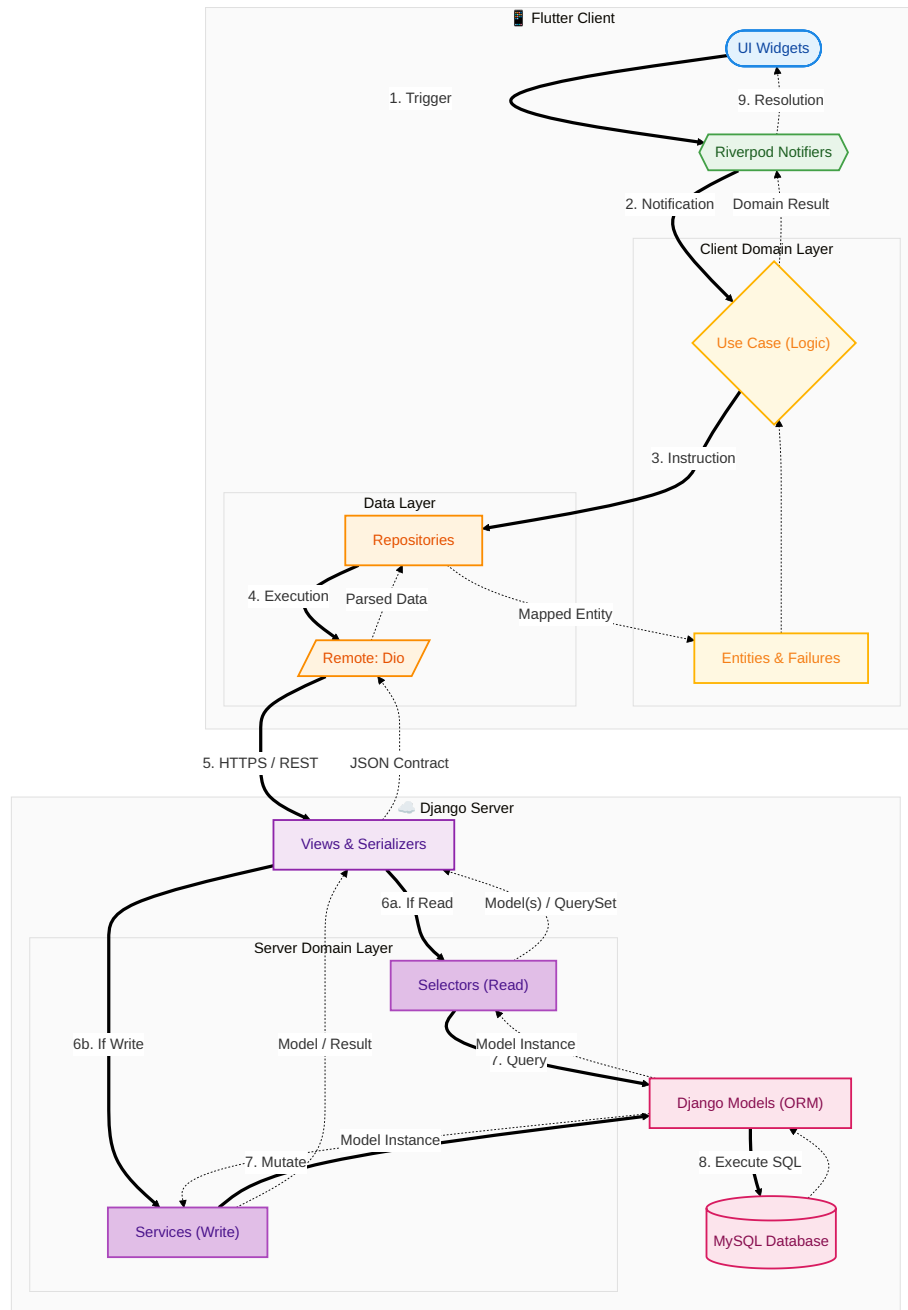


Figure 3.3: Data Propagation Flow from UI to Database

1. **User Trigger:** A technician interacts with a UI Widget to “accept booking”.
2. **State Management:** The Notifier catches the event and invokes the corresponding

Use Case.

3. **Repository Call:** The Use Case calls the Repository Contract, which is fulfilled by the Data Layer Implementation.
4. **Network Execution:** The Data Source executes a request to the Django Server.
5. **Server Persistence:** The Django API receives the request, processes the logic via a Service, and updates the MySQL Database.
6. **Response Mapping:** The result travels back through the layers, being mapped from JSON to an entity, updating the notifier state and the UI.

3.2 Behavioral Modeling

While the system architecture defines the static structure of the Karigar System, Behavioral Modeling defines the dynamic process and the flow of information across the platform. This section uses the Data Flow Diagram and the Sequence Diagrams to map out the logic, the transition of data and the interaction between various system actors.

3.2.1 Data Flow Analysis

The Data Flow Diagrams (DFDs) define the system boundaries and the internal decomposition of processes.

Context Diagram (Level 0). This diagram establishes the Karigar System as the central hub. It defines the interaction with external entities, including Customers, Technicians, and Admin while managing dependencies on third-party services JazzCash, Twilio and Google Maps.

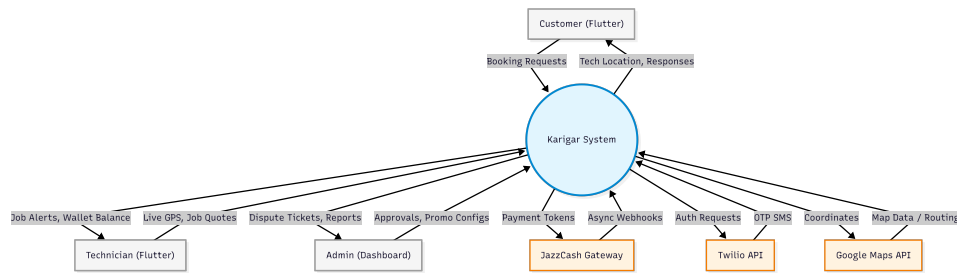


Figure 3.4: Context Diagram (Level 0 DFD)

Level 1 DFD (Process Decomposition). This provides a granular view of the five internal subsystems that drive the marketplace: Identity & Roles (1.0), Booking and Execution (2.0), Live Telemetry (3.0), Financial Ledger (4.0) and Support & Dispute (5.0).

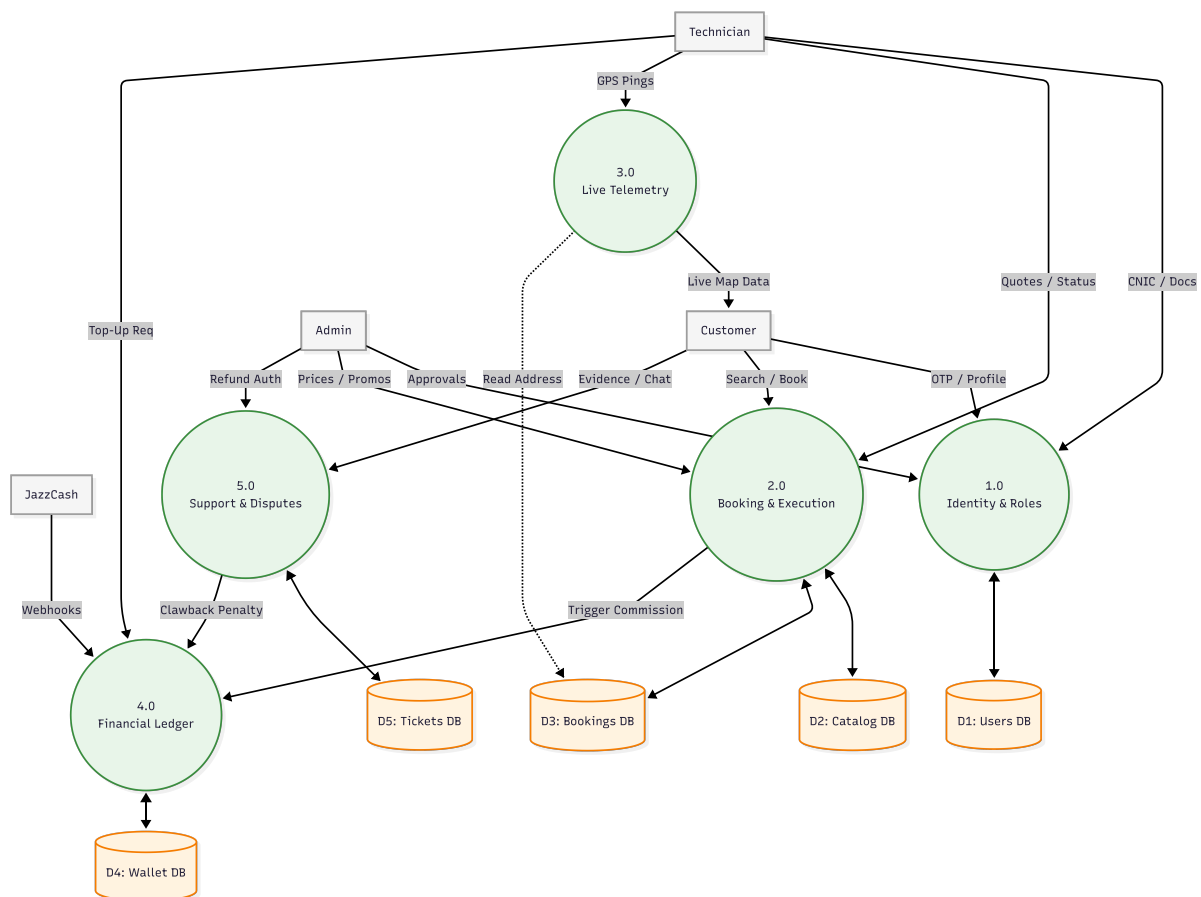


Figure 3.5: Level 1 DFD: Internal Process Decomposition

3.2.2 Sequence Diagrams

Sequence Diagrams are utilized to model the real-time communication protocols between the mobile clients and the backend API for critical business workflows.

3.2.2.1 The Multi-Phase Job Lifecycle

The system implements a structured sequence that moves through Inspection, Quotation, and Approval before final repair execution and commission deduction.

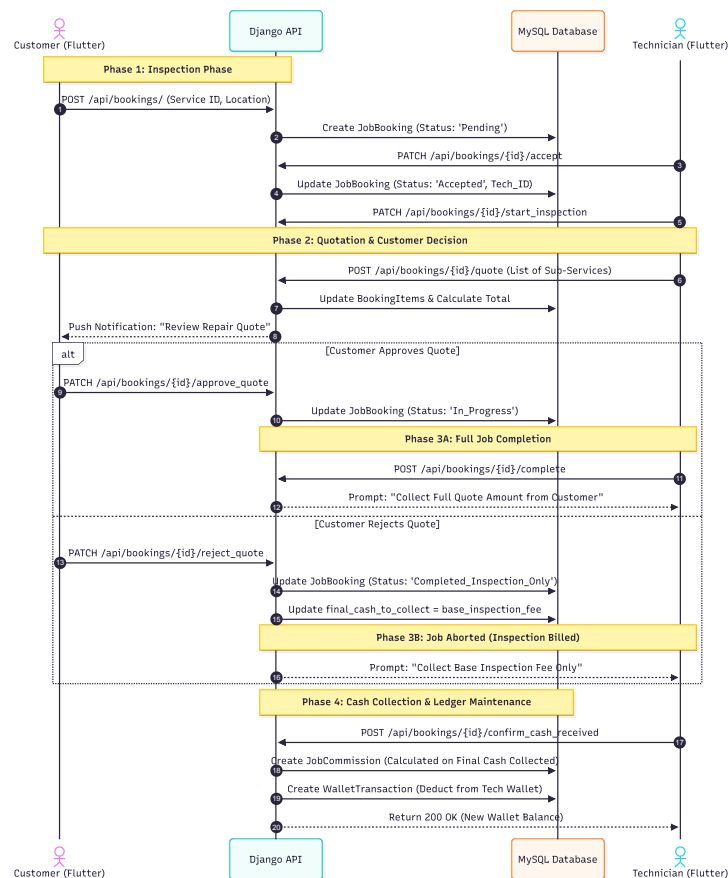


Figure 3.6: Multi-Phase Job Execution Sequence

3.2.2.2 Real-Time Tracking

The technician app sends GPS coordinates to a Redis cache every 5 seconds, which then broadcasts the location to the customer's map for a seamless experience.

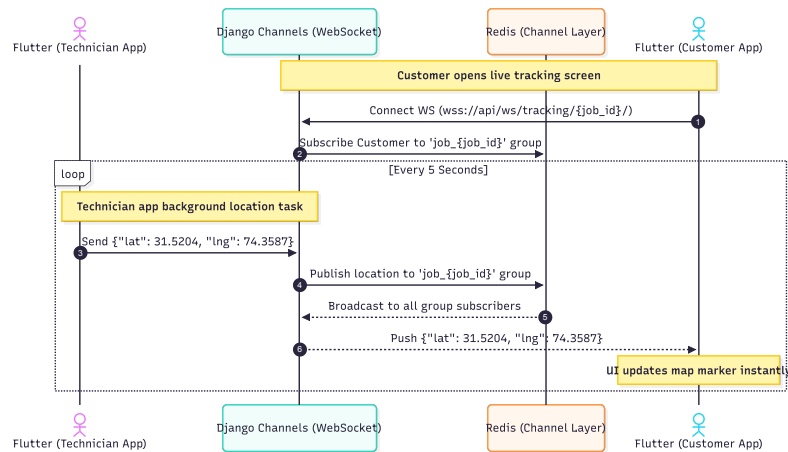


Figure 3.7: Real-Time GPS Broadcasting Sequence

3.2.2.3 Technician Top-Up

The top-up sequence shows the asynchronous webhook loop. It ensures that the Django Backend verifies the JazzCash Gateway signature before crediting the technician's virtual wallet.

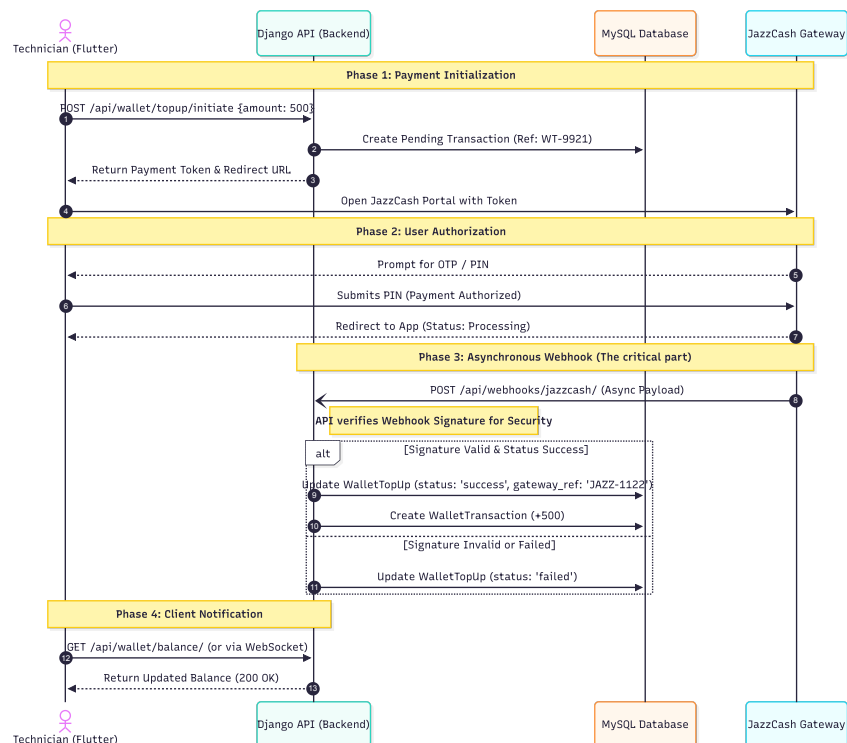


Figure 3.8: Secure Wallet Top-Up Sequence

3.2.2.4 Dispute Resolution

This flow models the process of Dispute resolution via an AI Chatbot. It concludes with a manual admin review executed as an ACID Database Transaction, ensuring that refunds and penalty deductions are committed as an atomic unit.

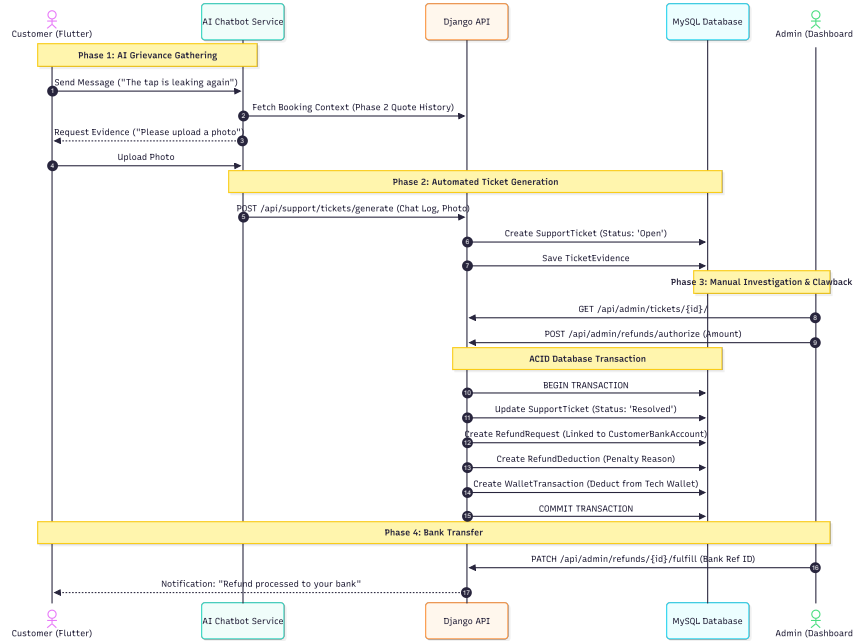


Figure 3.9: AI-Integrated Dispute and Refund Pipeline

3.3 Database Design

The Karigar System uses a normalized relational database schema. It is designed to maintain high data integrity across various modules. The architecture is optimized to support a multi-phase service marketplace.

3.3.1 Master Entity Relationship Diagram (ERD)

The Master ERD provides a comprehensive overview of the system's data architecture showing the relationship between identity management, service catalogs, operational booking and the financial ledgers. This diagram is highly pruned for understanding the core relationships; the detailed, per-module ERDs are presented in the subsections

that follow.

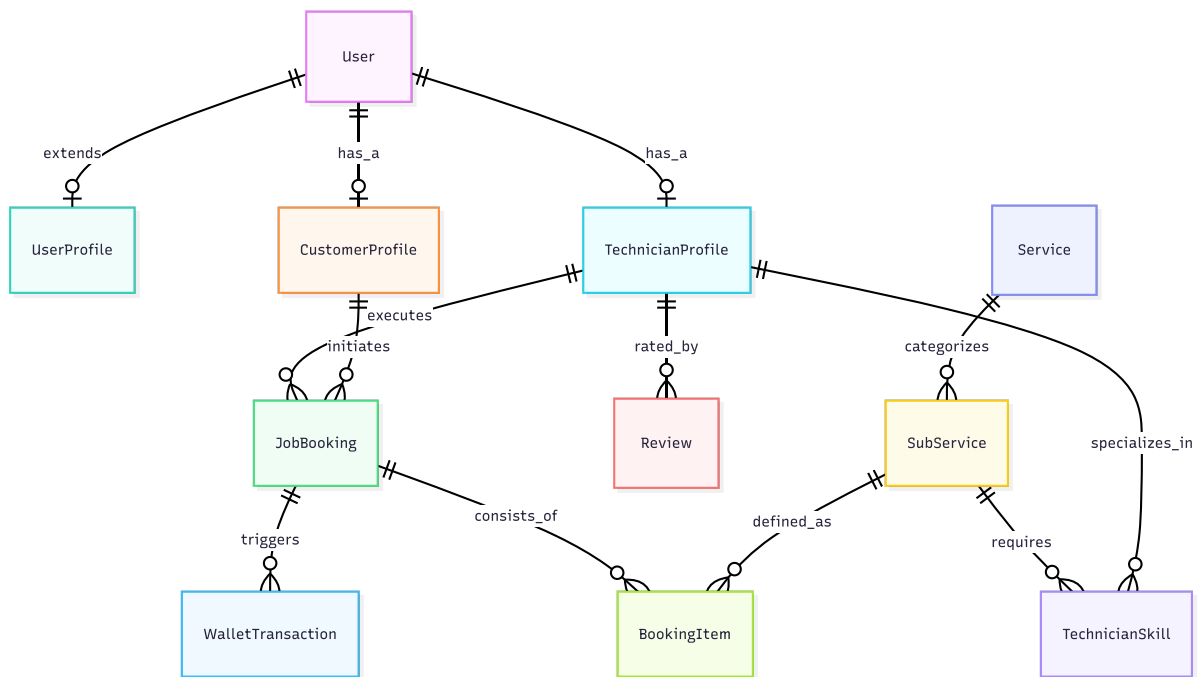


Figure 3.10: Master Entity Relationship Diagram

Core Logic. The schema has a centralized Django User entity that extends into specialized Customer and Technician profiles, which then interact through the Job Booking.

3.3.2 Modular Data Schemas

For clear architectural understanding the giant ERD is modularized into functional modules.

3.3.2.1 Identity and Role Management

This module manages the inheritance-based user model, addresses, and OTP-based authentication tokens.

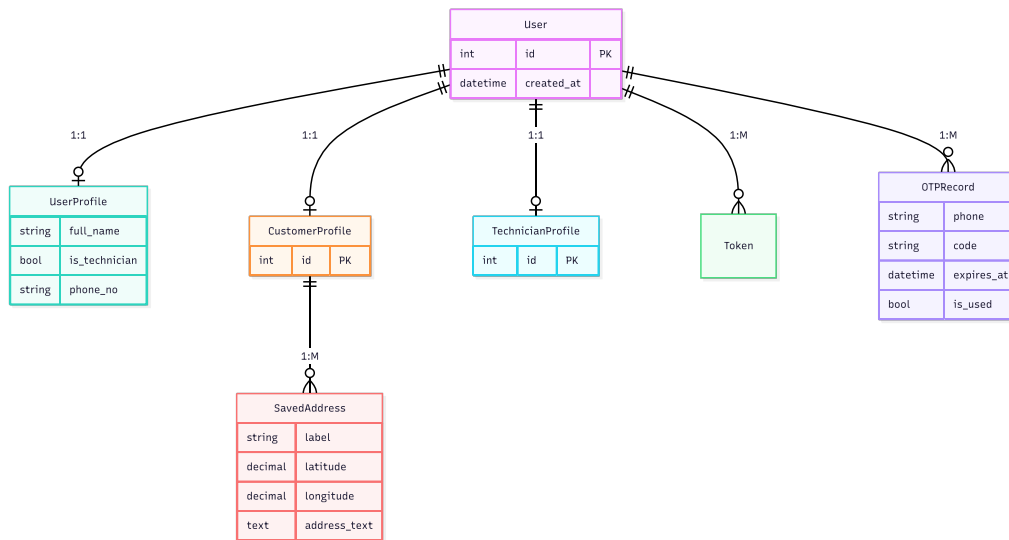


Figure 3.11: Identity and Access Schema

3.3.2.2 Technician Profiles and Scheduling

Handles technician-specific metadata, including professional skills, verification documentation, performance ratings, and availability calendars.

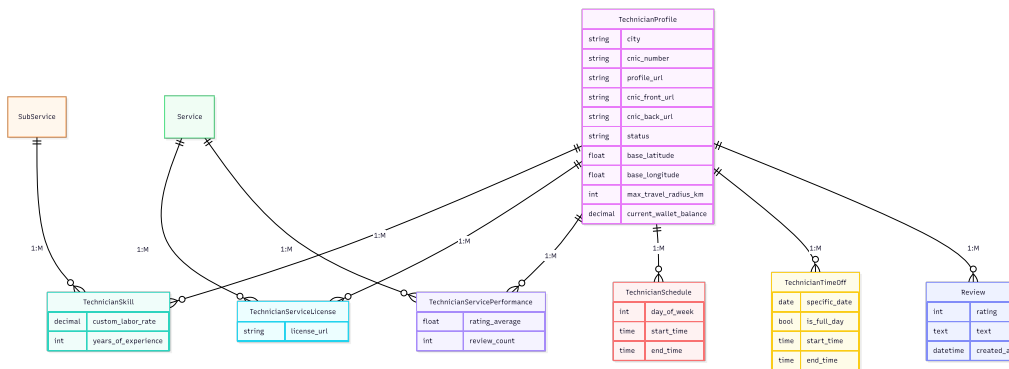


Figure 3.12: Technician Domain Schema

3.3.2.3 Service Catalog and Marketing

Defines the hierarchical structure of services and sub-services, coupled with promotional codes and featured marketing logic.

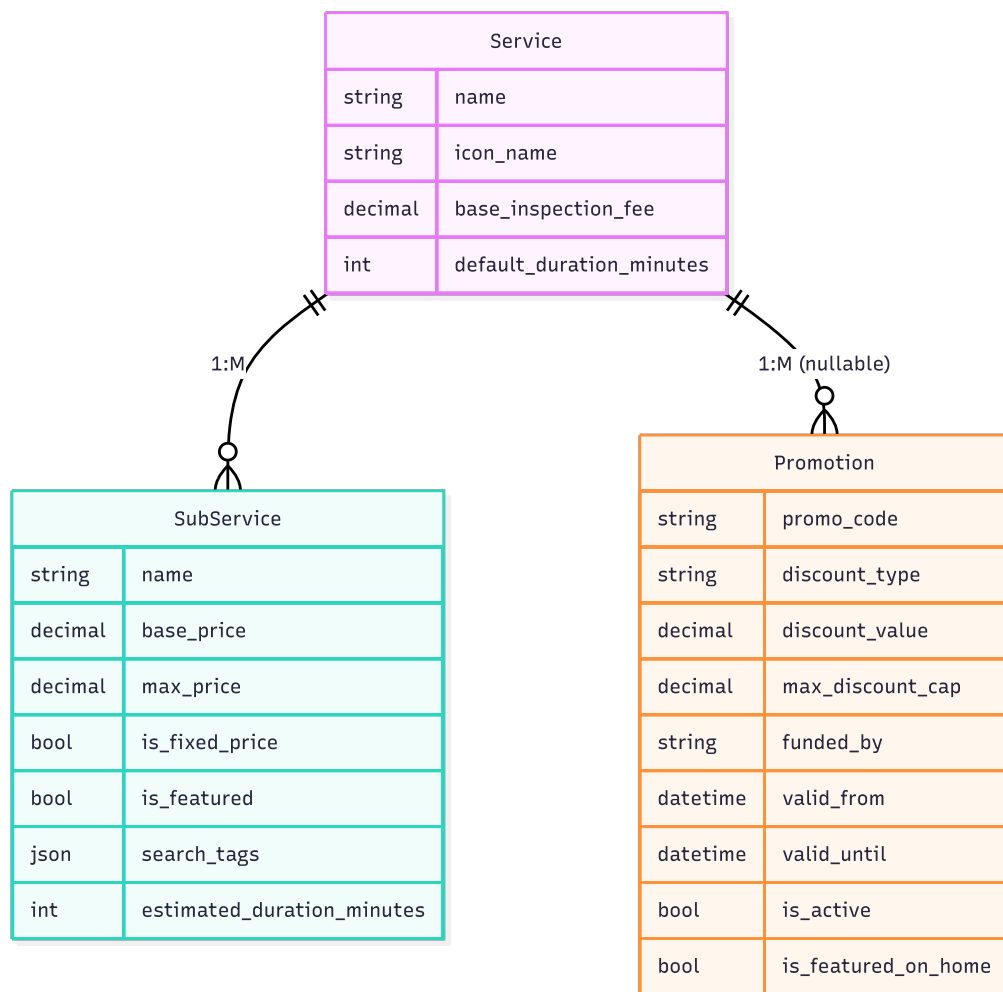


Figure 3.13: Catalog and Promotion Schema

3.3.2.4 Bookings and Field Operations

It manages the lifecycle of a job, tracking status transitions, service snapshots, and the linking of diagnostic inspections to repair tasks.

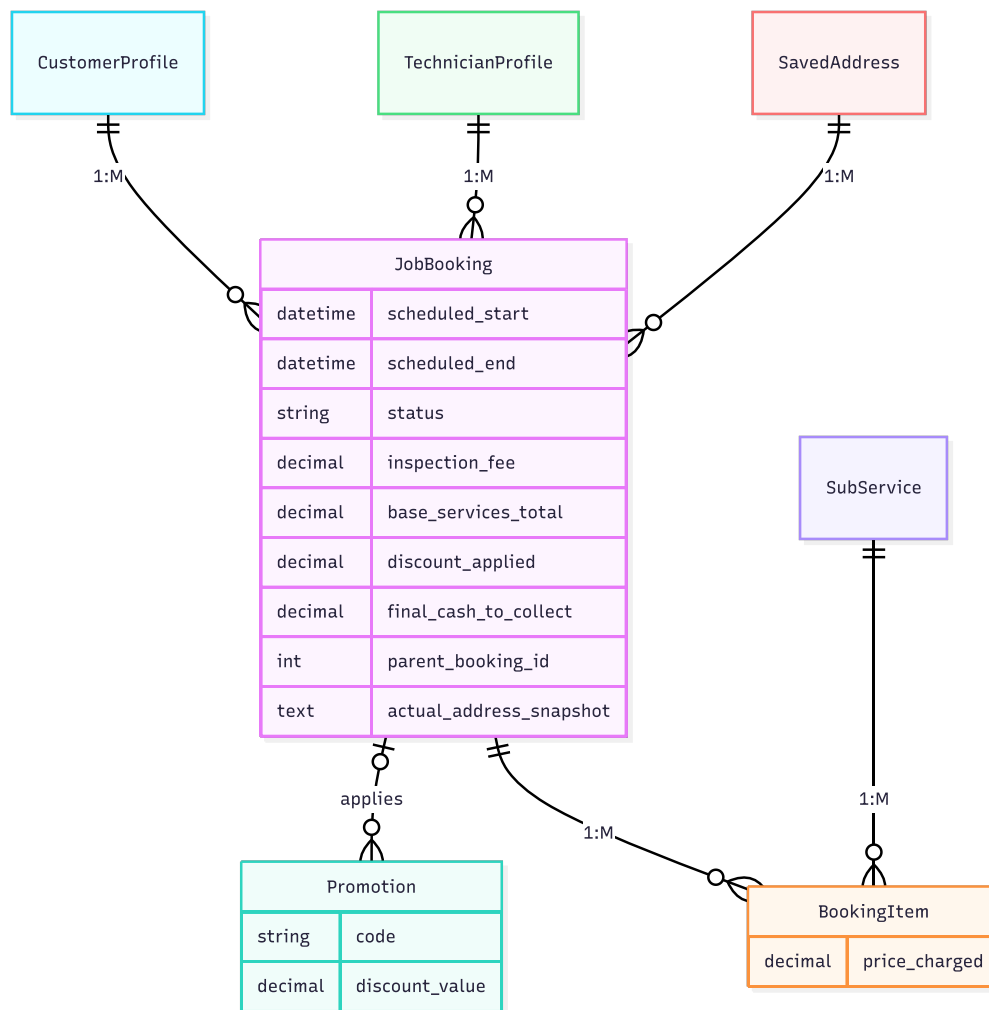


Figure 3.14: Operational Booking Schema

3.3.2.5 Financial Ledger and Support

Provides an immutable audit trail for wallet transactions, withdrawal requests, and evidence gathering for the dispute resolution pipeline.

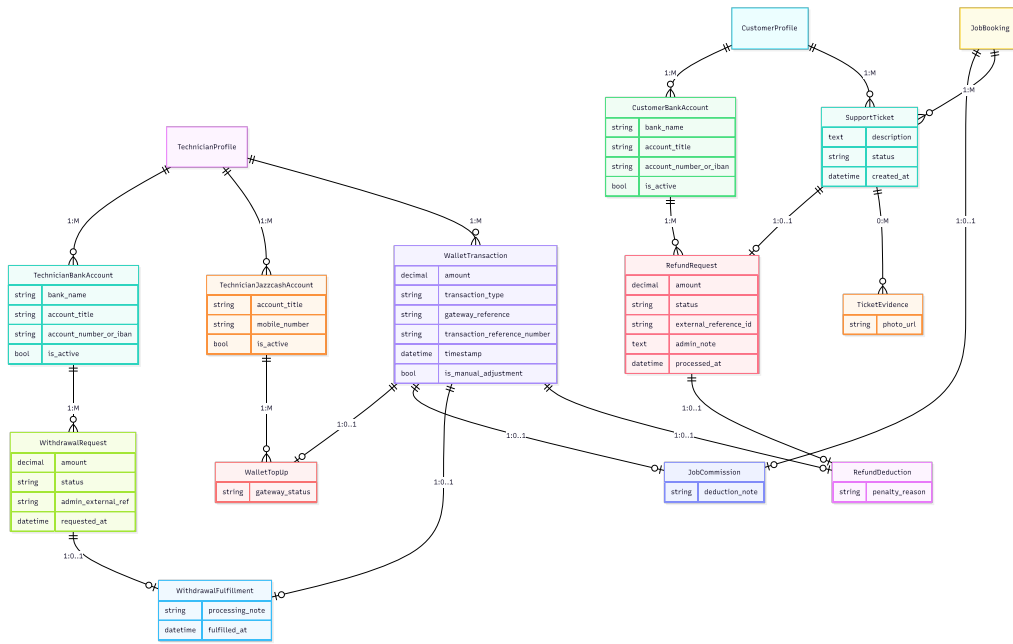


Figure 3.15: Financial and Support Schema

3.4 User Interface Design

The user interface (UI) of the Karigar System is designed to be simple, fast, and easy to use. Since the application is intended for both service providers (technicians) and customers in Pakistan, the design focuses on clarity and minimizing the number of steps required to complete a task.

3.4.1 Design Principles

The interface follows three core principles for professional and user-friendly experience.

- **Simplicity:** The screens are kept clean with minimal cognitive overload. This guarantees that even users who are not very technology intelligent can navigate through the app.
- **Consistency:** By using a unified design system, all buttons, icons, and text styles look the same throughout the app. This builds trust and reliability.
- **Responsiveness:** The UI is designed to work smoothly on a wide variety of mobile devices, ensuring that text is readable and buttons are easy to tap even on smaller

screens.

3.4.2 User Experience (UX) Strategy

The system is organized around the “Job Lifecycle” to make the process feel natural for the user:

- **Guided Navigation:** Users are guided through the two-phase booking process. Instead of overwhelming them with information, the app only shows what is necessary for the current step.
- **Visual Feedback:** The app provides instant feedback for every action. For example, when a user is searching, they see loading effects to show that data is being fetched, and “Snackbars” pop up to confirm if a task was successful or if an error occurred.
- **Safety and Trust:** Critical actions, like paying for a service or managing a wallet, are designed with clear confirmation steps to ensure financial security.

Chapter 4

Project Design

4.1 System Architecture

Explain the high-level architecture (e.g., Client-Server, MVC).

4.2 Database Design

Include your Entity-Relationship (ER) diagrams and database schemas.

4.3 User Interface Design

Showcase the wireframes or mockups of your application.

Chapter 5

Conclusion and Future Work

5.1 Conclusion

Summarize the successful development of the project and how it met the objectives outlined in Chapter 1.

5.2 Limitations

Acknowledge any constraints or features that were not fully realized.

5.3 Future Work

Suggest how this project could be expanded or improved in the future.

Bibliography

- [1] Author Name, *Title of the Book or Paper*, Publisher or Journal, Year.